An Innovative High-Speed Encryption Algorithm with Integrated Error Correction Capabilities

Panagiotis Andreadakis, Ioannis Stratakis, Petros Grafias

Word Count: 4480 words (main body) with 5 figures and 5 tables and 4 equations.

Article will submit to: *Journal of Cryptology*

Version: 1

Acknowledgments

We sincerely thank Gaming Laboratories International (GLI) for testing and certifying the RPC algorithm under the GLI-11 and GLI-19 standards. Their rigorous evaluation ensured compliance with igaming industry requirements, validating the algorithm's performance in randomness, encryption security and error correction. We also extend our deep gratitude to professor and researcher Periklis Papakonstantinou of Columbia University and Rutgers University for his invaluable cryptanalytic insights and contributions to the theoretical evaluation of RPC's security properties.

I. ABSTRACT

In an era where secure, high-speed encryption is crucial for IoT, cloud computing and real-time data processing, traditional cryptographic methods face challenges such as encryption latency and the lack of integrated error detection, correction and authentication. This paper introduces the Random Position Cipher (RPC), a novel cryptographic algorithm designed to address these limitations. RPC achieves high-speed encryption while integrating error detection, correction and authentication within a single encryption step. The algorithm operates using two dynamically permuted arrays and combines substitution, XOR operations, and array permutations to produce highly randomized outputs. This approach eliminates exploitable patterns and provides strong resistance to known cryptanalytic attacks. Performance evaluations show that RPC outperforms AES-NI and ChaCha20, achieving encryption speeds five to six times faster while providing enhanced randomness and built-in error correction.

Key words: Symmetric Encryption, Error Correction, Random Permutation, Cryptanalytic Resistance, Key Expansion, High-Speed Cipher, Post-Quantum Security.

II. INTRODUCTION

Over the past decades, cryptography has evolved significantly to meet the growing demand for secure systems in an increasingly digital world. Cryptographic algorithms play a critical role in ensuring the confidentiality, integrity, and authenticity of information across a wide range of applications—from personal communications to critical infrastructure protection [1].

Modern cryptographic methods are generally divided into two main categories: symmetric and asymmetric schemes, each with distinct advantages and trade-offs. Symmetric algorithms such as AES [2] and ChaCha20 [3] use a single key for both encryption and decryption. They offer high efficiency and low computational cost, making them ideal for encrypting large volumes of data. However, they pose challenges in secure key distribution and scalability, particularly in large or distributed systems [4]. Asymmetric algorithms, including RSA [5] and Elliptic Curve Cryptography (ECC) [6], use a public–private key pair to simplify key exchange. While they address the scalability issues of symmetric schemes, they are computationally intensive and significantly slower. Furthermore, asymmetric cryptosystems based on number-theoretic principles are especially vulnerable to quantum computing, which threatens their long-term security through algorithms like Shor's [7].

While symmetric algorithms such as AES offer a degree of quantum resistance through increased key sizes, this approach incurs greater computational overhead. Asymmetric algorithms such as RSA and ECC, however, are at significant risk due to their reliance on number-theoretic structures that can be efficiently broken using Shor's algorithm.

This paper introduces the Random Position Cipher (RPC), a novel symmetric cryptographic algorithm designed to address these limitations. Unlike RSA and ECC, RPC does not rely on algebraic or number-theoretic structures that quantum algorithms can exploit. Instead, it leverages random permutations and irreversible transformations to eliminate exploitable patterns, enhancing resistance to both classical and quantum attacks. A key innovation of RPC is the integration of error detection and correction mechanisms directly within the encryption process, features typically absent from conventional cryptosystems [8]. This makes RPC particularly well-suited for applications in error-prone environments such as satellite communications [9] and Internet of Things (IoT) networks. In addition to its strong security properties, RPC achieves high-speed encryption through computationally efficient transformations and high-entropy output. By resisting brute-force and quantum-based attacks while maintaining superior performance, RPC presents a promising solution for next-generation cryptographic applications.

This paper is structured as follows. **Section 1.** describes the encryption key initialization and key expansion process. **Section 2** describes the Random Position Cipher (RPC), analyzing the algorithm's key features in detail. **Section 3** discusses the time and computational performance of RPC. **Section 4** evaluates the randomness of RPC's output, while **Section 5** examines its security. Finally, **Section 6** concludes the paper.

1.0. THE RANDOM POSITION CIPHER

This section presents the Random Position Cipher (RPC), a novel cryptographic algorithm designed to integrate error correction and detection while ensuring high security and performance. The algorithm is based on three key arrays: K, D and D^{-1}, where K holds the key bytes, while D and D^{-1} serve as the core structures underpinning the algorithm's operation. The following content describes the algorithm initialization, the encryption and decryption mechanisms, along with the integrated error detection and correction process in detail.

1.1 The RPC initialization process

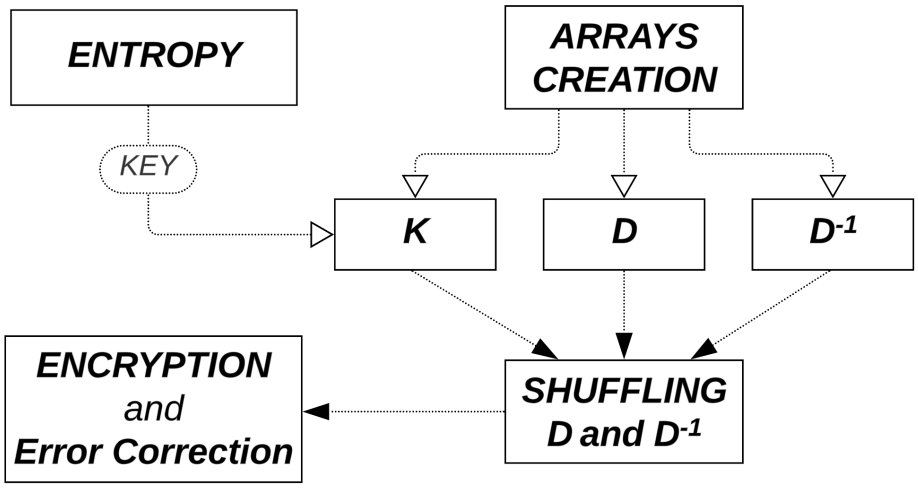

Fig. 1: Block diagram illustrating the fundamental steps and key elements of the RPC initialization process.

The initialization process, graphically summarized in Fig. 1, begins with the construction of the primitive key, which typically consists of 16 bytes or more, in accordance with the standards required for symmetric encryption algorithms. The key bytes are collected from an entropy source, such as thermal noise, quantum noise [10], or the operating system's entropy pool. The bits collected from one of these sources are used to construct the primitive key, which typically extends to 16 bytes or more, meeting the standards required for symmetric encryption algorithms. Subsequently, three critical arrays, namely K, D and D^{-1}, are defined. The Key bytes are stored in the vector K, while simultaneously the D and D^{-1} arrays are initialized with integers from 0 to 255 in sequential order. The three arrays at this point are as follows:

$$K = [K_0 , \dots , K_{15}]$$
$$D = [0 , \dots , 255]$$
$$D^{-1} = [0 , \dots , 255]$$

The next step of the initialization process is the key expansion [11], in which the initial 16-byte primitive key is expanded into a 256-byte key through 16 processing cycles applied to the fixed-length array K. The operations and basic working principle of each key expansion cycle are graphically summarized in Fig. 2. Prior to these 16 cycles, an additional warm-up cycle is performed. This step is essential for mitigating correlation-based key attacks, which exploit similarities between two keys used to encrypt the same plaintext. Even small differences in their bit patterns can lead to common elements in the resulting ciphertexts, potentially enabling key deduction or compromising the encryption. Moreover, the warm-up cycle enhances diffusion [12–14], ensuring that even minimal changes in the input keys produce significantly different outputs, consistent with the Avalanche Effect property [15–16].

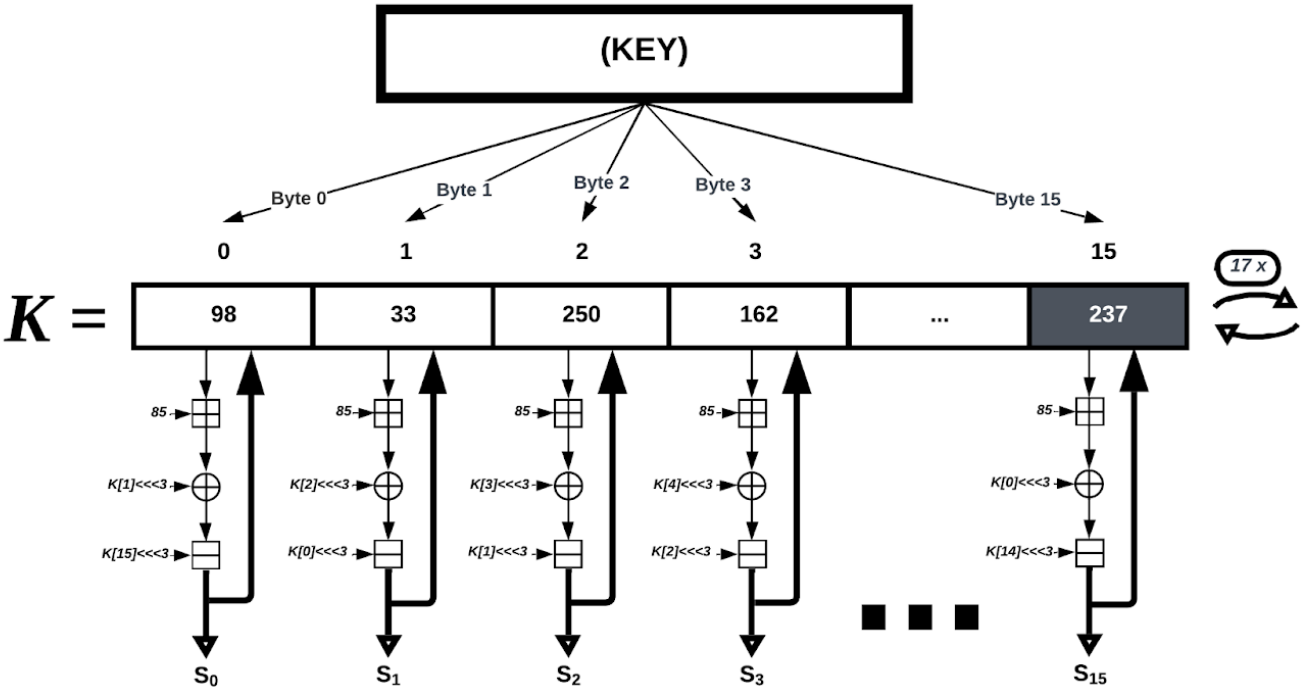

Fig. 2: Block diagram depicting the key expansion process, including the low-level operations involved.

The key expansion process is governed by the general formula:

$$S_i = [\ (K_i \boxplus 85) \oplus (K_{i+1} \lll 3)\ ] \boxminus (K_{i-1} \lll 3) \qquad (1)$$

Where $\boxplus$ represents the modular addition operator, $\oplus$ the XOR operator, $\lll 3$ indicates a three bit rotation to the left and finally $\boxminus$ is the modular subtraction operation. This formula highlights the inhomogeneous and non-linear nature of the key expansion process. Each operation within the cycle serves a specific purpose in enhancing the overall cryptographic strength, namely:

- Modular Addition ($\boxplus$): Introduces diffusion [17] through the use of the constant 85 (01010101), utilizing the carry effect [18] to propagate changes across the key bits.
- XOR ($\oplus$): Adds confusion [19] by altering the relationship between the current key byte and the next key byte in the array.
- Modular Subtraction ($\boxminus$): Enhances diffusion by propagating changes introduced in the previous key byte transformation.
- Rotation ($\lll$): Introduces non-linearity [20], further complicating the mixing pattern and increasing resistance to linear cryptanalysis.

During the key expansion process, two auxiliary vectors, D and D^{-1}, are updated using the Fisher–Yates shuffling technique [21–22]. This robust and widely adopted algorithm generates a random permutation of a finite sequence in an unbiased and efficient manner. It operates by iterating through a list and at each index, swapping the current element with another randomly selected from the remaining unshuffled portion.

In this implementation, the required randomness is derived directly from the pseudorandom indices S_0, S_1,…, S_{255}, obtained through the key expansion process. These indices are then used to perform the shuffle.

1.2. The RPC encryption process

The encryption step, which follows the initialization process, is illustrated below in Fig. 3.

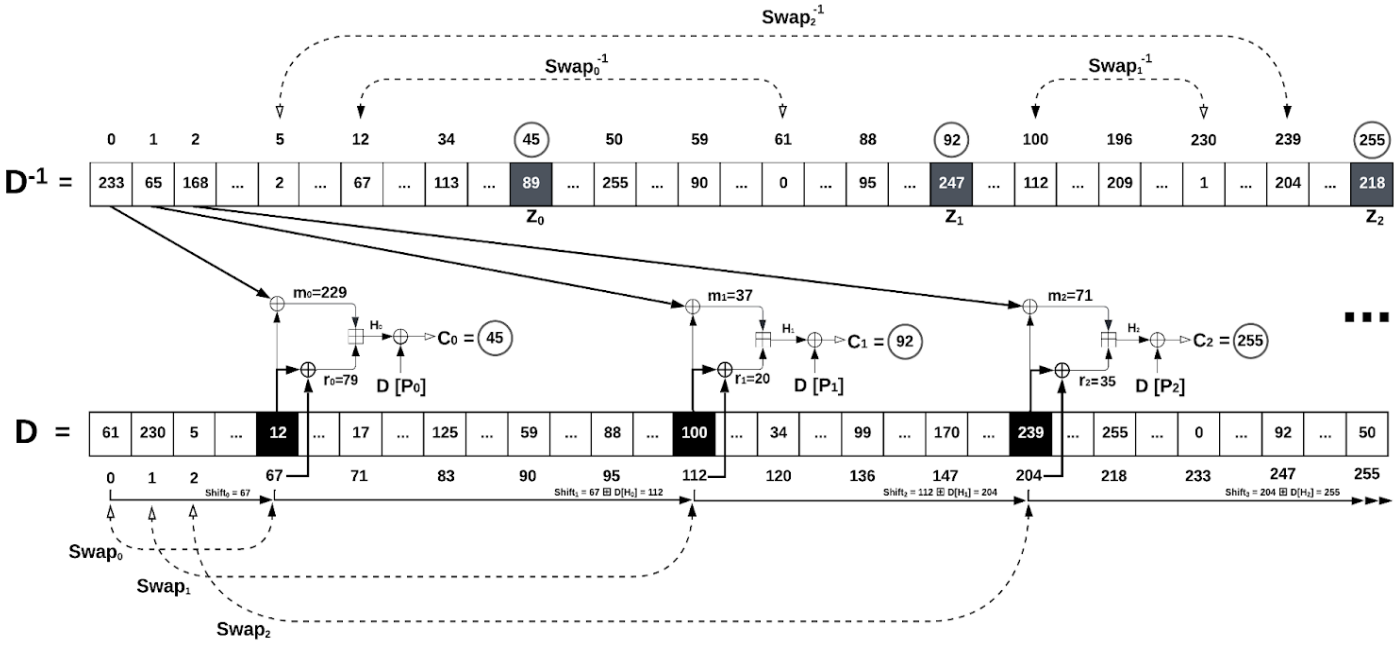

Fig. 3: Block diagram of the encryption process delineating the fundamental operations.

This step encompasses several fundamental operations, formulated as an iterative loop. The RPC algorithm repeatedly executes these steps to encrypt the subsequent characters $P_0, P_1, ..., P_n$, ensuring a continuous and secure encryption process across all data points. A practical example is presented in Table 1, illustrating both the core encryption operations and their corresponding numerical calculations in detail:

Table 1. Example: Encryption of the First Plaintext byte $P_0 = 5$

Encryption steps	**Calculation**
The initial encryption index, labeled as $Curr_0$, is determined by the final byte, S_{255}, generated during the Key Expansion Algorithm (see Fig. 2). For example, if $S_{255} = 67$, the encryption of the first plaintext character, P_0, takes place in cell $Curr_0 = S_{255} = 67$ of the array D.	$S_{255} = 67 \Rightarrow$ $\Rightarrow Curr_0 = S_{255} = 67$
Calculate the first pseudo-random value m_0 by utilizing the values of the arrays D and D^{-1}, as illustrated in Fig. 3:	$m_0 = (D^{-1}[0] \oplus D[Curr_0]) = (D^{-1}[0] \oplus D[67]) = (233 \oplus 12) = 229$
Determine the second pseudo-random value r_0:	$r_0 = (D[Curr_0] \oplus Curr_0) = (D[67] \oplus 67) = (12 \oplus 67) = 79$
Generate the next pseudo-random value H_0:	$H_0 = (m_0 \boxplus r_0) = (229 \boxplus 79) = 52$
Compute the masking value C_0 (e.g., for $P_0 = 5$):	$C_0 = (H_0 \oplus D[P_0]) = (H_0 \oplus D[5]) = (52 \oplus 18) = 38$
Translate C_0 through D^{-1} to obtain the final encrypted value Z_0:	$Z_0 = D^{-1}[C_0] = D^{-1}[38] = 89$
Execute swap in D:	$Swap_0(D[67] \rightleftarrows D[0])$
Execute swap in D^{-1}:	$Swap_0^{-1}(D^{-1}[D[67]] \rightleftarrows D^{-1}[D[0]])$
Slide to the next cell $Curr_1$ for the next encryption iteration:	$Curr_1 = Curr_0 \boxplus D[H_0]$

The Random Position Cipher carefully selects its internal operations to achieve a well-balanced combination of *diffusion* [17] and *confusion* [19], both of which are essential for resisting known cryptanalytic attacks. Table 2 summarizes the core operations used in RPC, highlighting the specific role each plays in enhancing either diffusion or confusion.

Table 2. Design Rationale for the Arithmetic Operations Used in RPC

Operation	Confusion	Diffusion	Justification
Modular Addition: $H_i = m_i \boxplus r_i$	No	Yes (Carry effect)	Introduces randomness as a mixing mechanism between cells in D and D^{-1}, thereby preventing direct inversion.
Substitution: $P_i \rightarrow D[P_i]$	Yes	No	Prevents direct correlation between plaintext and ciphertext.
XOR: $C_i = H_i \oplus D[P_i]$	Yes	No	Ensures that the combined effect of mixed randomness H_i and plaintext produces unpredictable ciphertext.
Substitution: $C_i \rightarrow D^{-1}[C_i]$	Yes	No	Prevents direct correlation between plaintext and C_i.
Swap in D: $Swap_i(D[Curr_i] \rightleftarrows D[i])$	No	Yes (State mutation)	Dynamically updates the permutation table D.
Swap in D^{-1}: $Swap_i^{-1}(D^{-1}[D[Curr_i]] \rightleftarrows D^{-1}[D[i]])$	No	Yes (State mutation)	Dynamically updates the permutation table D^{-1}.
Slide to next cell: $Curr_{i+1} = Curr_i \boxplus D[H_i]$	No	Yes (Permutation Shift)	Prevents fixed-position mappings, enhancing randomness.

All core operations of the RPC are selected to mitigate various forms of cryptanalytic attacks, including linear [23] and differential cryptanalysis [24], as well as known-plaintext attacks and frequency analysis [25]. Unlike traditional block ciphers that rely on static substitution tables (e.g., AES S-boxes), RPC dynamically evolves both its substitution and permutation states. This dynamic behavior enhances unpredictability and significantly increases the cipher's resilience against cryptanalysis. A more detailed exploration of this concept is provided in **Section 5.0**.

1.3. The RPC decryption process

The decryption process mirrors the encryption steps, utilizing analogous operations to reconstruct the original plaintext from the ciphertext. This process is structured into the following key steps:

- Decrypt Operation: Each plaintext byte P_i is reconstructed by the operation $P_i = D^{-1}[(m_i \boxplus r_i) \oplus D[Z_i]]$, which inversely applies the transformations used during encryption.
- Swap in D: The swap function $Swap_i$ modifies the permutation of array D by interchanging the elements at positions $Curr_i$ and i, $Swap_i(D[Curr_i] \rightleftarrows D[i])$ specifically.
- Swap in D^{-1}: Similarly, $Swap_i^{-1}$ alters the permutation of the inverse array D^{-1} by swapping the elements at positions derived from $D[Curr_i]$ and $D[i]$, executed as $Swap_i^{-1}(D^{-1}[D[Curr_i]] \rightleftarrows D^{-1}[D[i]])$.
- Slide to the Next Cell in D: The index for the next cell, $Curr_{i+1}$, is updated using the slide operation, calculated as $Curr_{i+1} = Curr_i \boxplus D[H_i]$. This step adjusts the current processing point to the next relevant position in the permutation array D, ensuring the sequence continuity for the subsequent decryption iteration.

These steps are applied iteratively to each byte of the ciphertext, systematically recovering the original plaintext sequence.

2.0. The RPC error detection process

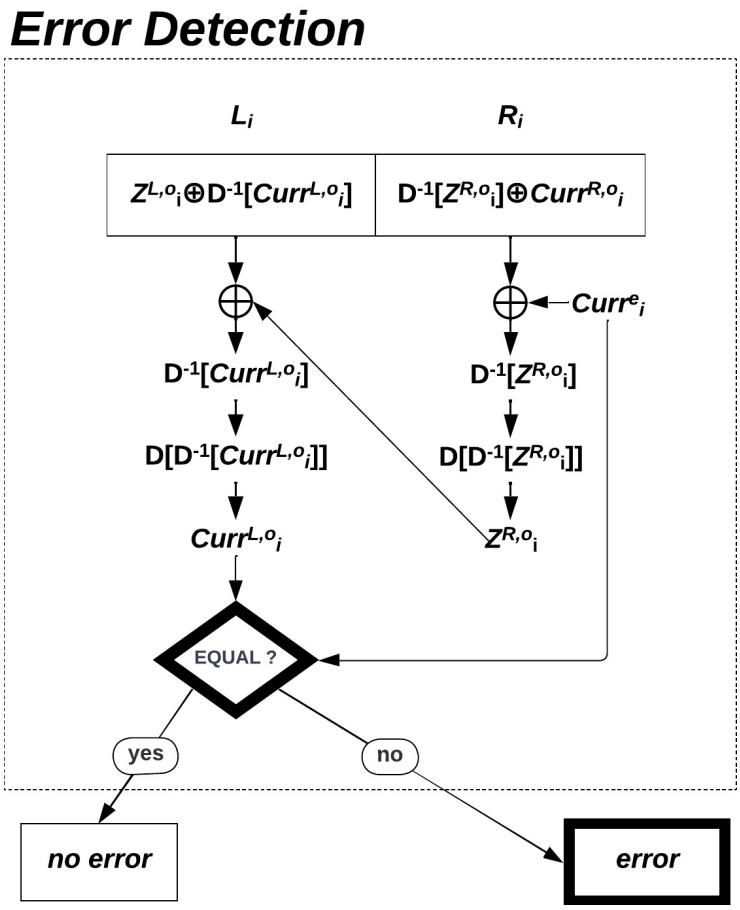

Fig. 4: Block diagram of the error detection key steps and operations.

The error detection process, as depicted in Fig. 4., is a fundamental feature of the RPC algorithm. It enables the identification of any errors in the encrypted plaintext $P^o{}_i$. The process begins with the generation of codewords (L_i , R_i), representing the left-hand and right-hand components, respectively, of each codeword generated by the sender. Every 16-bit codeword (L_i , R_i) corresponds to the associated plaintext $P^o{}_i$. The left-hand component, L_i, is derived using Eq. (2). In this step, the ciphertext $Z^o{}_i$, representing the encryption of the 1-byte plaintext $P^o{}_i$, is XORed with $D^{-1}[Curr^o{}_i]$, where $Curr^o{}_i$ indicates the current index of the array D. The superscript "o" denotes the observed value produced during the encryption step. Similarly, the right-hand component, R_i, is computed using $Z^o{}_i$, which is XORed with $D^{-1}[Z^o{}_i]$ and $Curr^o{}_i$ to produce the right half R_i, as shown in Eq. 3.

$$L_i = Z^o{}_i \oplus D^{-1}[Curr^o{}_i] \qquad (2)$$

$$R_i = D^{-1}[Z^o{}_i] \oplus Curr^o{}_i \qquad (3)$$

Once the codeword components L_i and R_i are generated, they are transmitted to the receiver. The receiver then performs the error detection process, which involves a series of operations summarized in the following steps:

1. $R_i \oplus Curr^e_i = D^{-1}[Z^{R,o_i}] \oplus Curr^{R,o_i} \oplus Curr^e_i = D^{-1}[Z^{R,o_i}] \Rightarrow Z^{R,o_i}$
2. $L_i \oplus Z^{R,o_i} = Z^{L,o_i} \oplus D^{-1}[Curr^{L,o_i}] \oplus Z^{R,o_i} = D^{-1}[Curr^{L,o_i}] \Rightarrow Curr^{L,o_i}$
3. *If* $Curr^e_i = Curr^{L,o_i} \Rightarrow$ *no detected error.*
4. *If* $Curr^e_i \neq Curr^{L,o_i} \Rightarrow$ *error detected.*

The superscripts "L" and "R" refer to the left and right values, respectively, indicating the two components of the codeword associated with each plaintext P_i. For each plaintext, the receiver receives a codeword comprising two parts, L_i and R_i, which are used to derive the corresponding *Curr* and Z values. Furthermore, the superscript "e" denotes the expected correct value, distinguishing it from the superscript "o," which, as previously defined, represents the observed value. One key thing is that wherever the RPC's Error Correcting Code (ECC) is employed, a recursive slide is performed: $Curr_{i+1} = Curr_i \boxplus (D[H_i] \oplus P_i)$.

2.1. The RPC error correction process

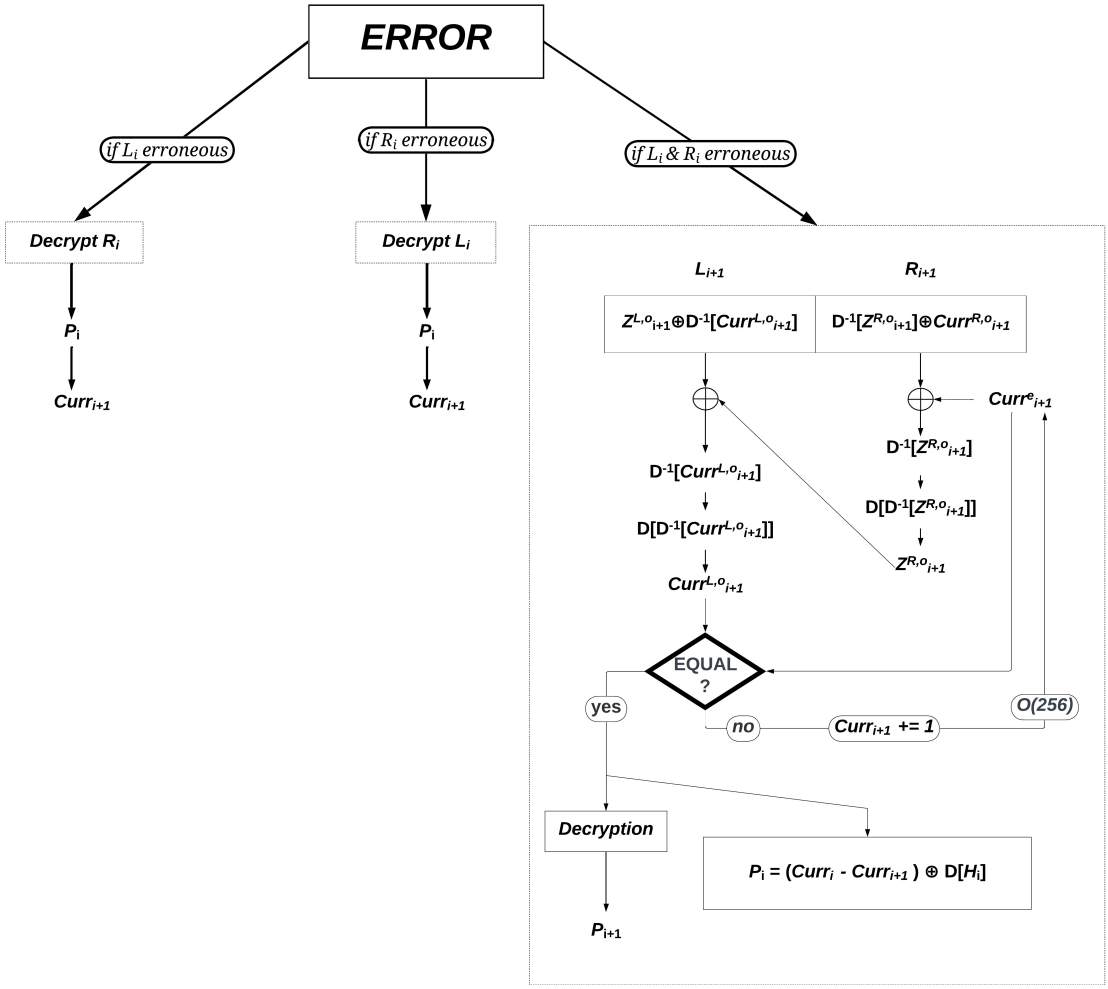

Fig. 5: Block diagram of the error correction key steps and operations

The objective of the error correction process, illustrated in Fig. 5, is to resolve the alarm triggered by the error detection mechanism described in **Section 2.0.** This is achieved by identifying the error

location and restoring the erroneous codeword. The process accounts for three distinct error scenarios, outlined below:

- If L_i is erroneous and Ri is correct, the algorithm attemts to recover the correct plaintext byte P^{o}_i, using the R_i *code*, assuming the subsequent codeword (L_{i+1} , R_{i+1}) *is valid:*

$$R_i \oplus Curr^{e}_i = D^{-1}[Z^{R,o}_i] \oplus Curr^{R,o}_i \oplus Curr^{e}_i = D^{-1}[Z^{R,o}_i] \Rightarrow Z^{R,o}_i \Rightarrow P^{o}_i$$

- If R_i is erroneous and L_i is correct, the algorithm computes the correct byte P_i, using the L_i *code,* again assuming the next codeword (L_{i+1} , R_{i+1}) *is valid:*

$$L_i \oplus D^{-1}[Curr^{e}_i] = Z^{o}_i \oplus D^{-1}[Curr^{o}_i] \oplus D^{-1}[Curr^{e}_i] = Z^{o}_i \Rightarrow P^{o}_i$$

- If both L_i and R_i are erroneous, the algorithm resorts to brute-force (complexity O(256)) proceeds to find the correct byte P^{o}_i, replying on the validity of the next codeword (L_{i+1} , R_{i+1}).

In the first two scenarios, the correction algorithm decrypts the two candidate ciphertexts $Z^{L,o}_i$ and $Z^{R,o}_i$, retrieving the corresponding plaintext values $P^{L,o}_i$ and $P^{R,o}_i$. From these, the next-step values $Curr^{L,o}_{i+1}$ left and $Curr^{R,o}_{i+1}$ right are derived. These are then used to derive the next-step positions.

The error detection mechanism (see **Section 2.0.**) is then applied to the subsequent codeword, L_{i+1} R_{i+1}, and the valid decryption path is determined based on which candidate does not trigger an error. The algorithm determines which of the two decrypted options does not trigger errors, ensuring the correct choice.

In the third scenario, where neither $Z^{L,o}_i$ nor $Z^{R,o}_i$ yields a valid result, the algorithm systematically tests all possible combinations of P_i and $Curr_{i+1}$, identifying the correct one based on which produces a valid codeword (L_{i+1} , R_{i+1}). The interleaving strategy plays a crucial role here by reducing the probability of consecutive codeword errors, thereby allowing reliable correction even in these challenging cases.

In rare cases, both L_i and R_i may be erroneous and the error may go undetected [26], however, the probability of such an event is extremely low. To mitigate this risk, the RPC error detection algorithm relies on the assumption that adjacent codewords, such as (L_i , R_i) and (L_{i+1} , R_{i+1}), are unlikely to be simultaneously corrupted. Furthermore, because each codeword depends on its predecessor, any undetected error is likely to propagate through subsequent cycles, thereby increasing the chance of detection over time. When such errors occur, the algorithm employs brute-force correction to identify and restore the original erroneous codewords, ensuring continued system reliability. Additionally, interleaving strategies [27], adapted from prior algorithmic frameworks, further reduce the likelihood of consecutive codeword corruption, reinforcing the overall robustness of the RPC algorithm.

3.0. TIME AND COMPUTATIONAL PERFORMANCE

3.1. Speed of RPC

This section provides a detailed performance analysis of the RPC encryption system, implemented in C++, compared to the AES-NI-128-CTR [28] and ChaCha20-SIMD [29] algorithms, both executed within an OpenSSL environment [30]. To ensure consistency and reliability, all evaluations were conducted on a single core of an Intel i7-8750H processor running at 2.2 GHz.

The study measured encryption and decryption times across various file sizes, including 5MB, 100MB, 500MB, 1000MB, 2000MB, and 4000MB. The results, summarized in Table 3, illustrate the relative performance of the RPC system in terms of encryption and decryption speed compared to the widely used AES-NI-128-CTR and ChaCha20-SIMD algorithms. The table demonstrates that the RPC outperforms its main competitors across all considered file dimensions, despite being implemented in C++, which is slower than the C implementation used for the other two cases in OpenSSL.

Table 3. Performance for Encryption and decryption time in total of RPC, AES and ChaCha20

	RPC (*C++*)	AES-NI-128 OpenSSL (*C*)	ChaCha20 SIMD OpenSSL (*C*)
File size	**Enc/Dec Time**	**Enc/Dec Time**	**Enc/Dec Time**
10 MB	8ms	49ms	44ms
200 MB	175ms	430ms	518ms
1000 MB	910ms	2134ms	2246ms
2000 MB	1815ms	4627ms	4862ms
4000 MB	3633ms	8996ms	9544ms

3.2. Time Complexity of RPC Error-Correcting Codes

The RPC algorithm exhibits exceptional efficiency in its core functions, with encoding, decoding, and error detection all operating in constant time, O(1), regardless of the codeword length. This efficiency stems from RPC's fundamental design, which relies on a simple XOR operation to compute the two halves of each codeword, as outlined in Eqs. (2) – (3). Since both L_i and R_i are derived using a single XOR operation, encoding and decoding remain O(1), making RPC significantly faster than traditional error-correcting schemes.

Similarly, error detection in RPC is also O(1), requiring only one XOR operation and a direct comparison between the values of $Curr^{e}_{i}$ and $Curr^{L,o}{}_{i}$, as previously described (see **Section 3.0**). This eliminates the need for complex iterative or probabilistic checks.

Error correction, however, varies between O(1) and O(256) in the worst case scenario. If an error affects only one half of the codeword (L_i or R_i), correction remains O(1). However, if both halves are corrupted, an exhaustive search across 256 possible byte values may be required to reconstruct the correct sequence. In practice, the actual computational cost is often much lower than O(256), as the correct value is frequently identified early in the search. Furthermore, leveraging 256 GPU cores allows all possible values to be checked simultaneously, effectively reducing the correction time to O(1) in all cases. The likelihood of both halves of a codeword being corrupted simultaneously is extremely low in real-world noise environments. As discussed in **Section 3.1.**, interleaving further mitigates this risk, making such occurrences virtually nonexistent in practical applications.

Compared to conventional error-correcting codes such as Reed-Solomon [31], BCH [32], and LDPC [33], which typically have a time complexity ranging from O(cw) to O(cw^2) for decoding and correction, particularly in burst-error scenarios [34-35], RPC offers a significantly lower computational overhead. Traditional schemes rely on matrix operations, Galois field arithmetic [36-37], or iterative belief propagation, all of which are computationally intensive. Even in its worst-case scenario, RPC's brute-force search over 256 possible values per corrupted byte is far less demanding than solving large systems of equations or performing multiple rounds of iterative decoding. By achieving O(1) encoding, decoding and error detection, along with a worst-case O(256) error correction, RPC delivers an exceptionally efficient error-correction approach that outperforms traditional methods in computational simplicity.

3.3. RPC's period estimation

The period of the RPC cipher is estimated experimentally by analyzing cycle lengths in smaller internal states and extrapolating trends to larger configurations. The process begins with an RPC-4 state, where the permutation arrays D and D^{-1} each contain $n = 4$ cells storing 2-bit codewords. Encrypting a sequence of plaintexts consisting solely of zeros in this setting results in an observed period of approximately $\sim 2^2$ iterations. To generalize this estimation for larger values of n (up to 16), the empirical growth pattern of observed cycle lengths is analyzed and compared with trends in factorial

expansion. While the total number of unique permutations follows $n!$ growth, practical cycle lengths are shorter due to reversibility constraints in the system, which limit the number of effective unique permutations. The observed data align with a sub-factorial reduction model, leading to the derivation of an approximation:

$$\mathrm{RPC}_{period} = \frac{n!}{\sqrt[3]{n!}} \qquad (4)$$

This function accounts for factorial state complexity while incorporating correction terms obtained from experimental measurements, providing a reliable upper bound for the estimated period. The scaling pattern was established by analyzing values of n from 4 to 16 and fitting the data to a model that reflects factorial growth while considering internal constraints. Table 4. presents estimated period values for $n > 16$, extrapolated using the Eq. (4).

Table 4. Estimated Period of RPC for Different State Sizes

Number of cells	Number of iterations
4	$\sim 2^{2}$
8	$\sim 2^{10}$
16	$\sim 2^{30}$
32	$\sim 2^{79}$
64	$\sim 2^{198}$
128	$\sim 2^{478}$
256	$\sim 2^{1,122}$

This factorial-based growth confirms that the cycle length increases super-exponentially with n, making periodic-based attacks infeasible. Since the observed cycle for small n follows an arithmetic progression relative to factorial scaling, the derived formula provides a reliable upper bound for realistic implementations.

4.0. EVALUATION OF THE RPC RANDOMNESS

4.1. NIST STS-SP800/22 statistical test suite results

The randomness evaluation was conducted on encrypted sequences of zero plaintexts, utilizing both non-scrambled initial D and D^{-1} arrays. This assessment followed the NIST STS-SP800/22 standard [38], which applies a suite of statistical tests to measure pseudorandomness. Specifically, the evaluation focused on the sequences Z_i, H_i and S_i, where S_i corresponds to the outputs of the key expansion algorithm described in **Section 1.1.**

Additionally, the error correction and detection codewords L_iR_i were analyzed. Their unpredictability primarily arises from the variables Z_i and $Curr_i$, which are influenced by the permutations of D and D^{-1}. These permutations, in turn, are determined by the mixing values H_i. Each statistical test was executed multiple times in a loop, using 10 binary sequences generated from 10 different RPC arrays, with each sequence containing 12 million bytes. The evaluation framework begins by outlining the test methodology and defining the specific defects each statistical test aims to detect. The NIST framework employs hypothesis testing, a statistical approach that determines whether an observed sequence exhibits characteristics consistent with true randomness. In this context, the tests examine whether the

binary sequences produced by RPC encryption significantly deviate from expected random distributions.

Table 5 presents a detailed, step-by-step overview of the procedure used to evaluate the randomness of the binary sequences. To correctly interpret the results in Table 5, the values presented (e.g., 0.1, 0.5, 0.8) represent p-values, which indicate whether a given statistical test accepts or rejects the hypothesis that the data is random. These values range from 0 to 1. A p-value close to 1 (e.g., 0.83) means that the test found no deviation from randomness, whereas a p-value below 0.01 would suggest a potential failure in randomness. In Table 5, all tests resulted in p-values within acceptable ranges and are marked as "PASS," confirming that the RPC encryption successfully meets the randomness criteria defined by the NIST STS-SP800/22 standard.

Table 5. NIST STS SP-800/22 results for the sequences Z_i, H_i, S_i and L_iR_i

Randomness type test	*P-values of* Z_i	*P-values of* H_i	*P-values of* S_i	*P-values of* $L_i R_i$
Frequency test (MONOBIT)	*0.81 PASS*	*0.73 PASS*	*0.35 PASS*	*0.80 PASS*
Frequency test within block	*0.29 PASS*	*0.66 PASS*	*0.35 PASS*	*0.27 PASS*
Run test	*0.65 PASS*	*0.55 PASS*	*0.62 PASS*	*0.88 PASS*
Longest Run of ones in a block	*0.77 PASS*	*0.79 PASS*	*0.90 PASS*	*0.42 PASS*
Binary matrix Rank test	*0.74 PASS*	*0.32 PASS*	*0.12 PASS*	*0.69 PASS*
Discrete Fourier Transform (Spectral) Test	*0.55 PASS*	*0.24 PASS*	*0.28 PASS*	*0.59 PASS*
Non-Overlapping Template Matching Test	*0.42 PASS*	*0.81 PASS*	*0.60 PASS*	*0.64 PASS*
Overlapping Template Matching Test	*0.64 PASS*	*0.54 PASS*	*0.91 PASS*	*0.19 PASS*
Maurer's Universal Statistical Test	*0.19 PASS*	*0.87 PASS*	*0.53 PASS*	*0.70 PASS*
Linear Complexity Test	*0.21 PASS*	*0.23 PASS*	*0.45 PASS*	*0.78 PASS*
Serial Test	*0.36 PASS*	*0.21 PASS*	*0.73 PASS*	*0.35 PASS*
Approximate Entropy Test	*0.21 PASS*	*0.16 PASS*	*0.53 PASS*	*0.35 PASS*
Cumulative Sums (Forward) Test	*0.89 PASS*	*0.59 PASS*	*0.35 PASS*	*0.91 PASS*
Cumulative Sums (Reverse) Test	*0.84 PASS*	*0.55 PASS*	*0.21 PASS*	*0.69 PASS*
Random Excursions Test	*0.75 PASS*	*0.48 PASS*	*0.59 PASS*	*0.35 PASS*
Random Excursions Variant Test	*0.39 PASS*	*0.17 PASS*	*0.88 PASS*	*0.35 PASS*

4.2. PractRand statistical test suite results

The RPC algorithm underwent additional rigorous randomness evaluations using PractRand [39] under the worst-case scenario: continuously encrypting zero plaintext for 51 days, starting from the initial non-scrambled D and D^{-1} arrays, to assess the pseudorandomness of the sequence Z_i.

PractRand is a comprehensive testing suite that applies a wide range of advanced statistical tests to evaluate the quality of pseudorandom numbers generated by a cipher or algorithm. The RPC algorithm successfully passed all PractRand randomness tests, even when restricted to an 8-bit output, without exhibiting any anomalies. It maintained strong randomness over a cipher stream of up to 1 PB (petabyte).

In comparison, other algorithms struggled under similar conditions: RC4 [40] failed after 1 TB (terabyte), while ChaCha20 failed after 256 GB (gigabytes) in the same worst-case encryption scenario [41].

4.3. RPC's global certifications

The RPC algorithm has earned two significant certifications GLI-11 [42] and GLI-19 [43] which are crucial in regulated industries. GLI-11 certifies the pseudorandomness and fairness of gaming devices and systems, ensuring that RPC generates unbiased, high-quality pseudorandom numbers in compliance with strict industry standards. GLI-19 focuses on interactive gaming systems, evaluating the reliability and integrity of pseudorandom number generation in real-time platforms.

These certifications were granted to both RPC-256 and RPC-64, a scaled-down variant featuring 64 cells in each D and D^{-1} array, with each cell storing 6-bit codewords. Notably, RPC-64 passed certification under worst-case conditions, demonstrating that the RPC algorithm maintains a high standard of pseudorandom number generation even with a constrained internal state.

5.0. SECURITY EVALUATION OF RPC

5.1. General security proofs

G-statement: The RPC system is secure against cryptanalysis if and only if the permutations of the arrays D and D^{-1} are both pseudorandom and secret.

Proof of G-statement: The security of RPC is fundamentally tied to the indistinguishability of the permutations in D and D^{-1} from true randomness. If an attacker cannot discern their structure or predict future encryption states, the system remains secure. These permutations are governed by the sequence of values H_i, which control the sliding and transformation operations. The randomness of H_i has been rigorously validated using the NIST STS/SP800-22 statistical test suite, even in a worst-case scenario where zero plaintext is encrypted with an initially unscrambled D and D^{-1}. These results confirm that the induced permutations exhibit strong pseudorandom properties, preventing statistical inference. The recursive nature of RPC encryption is defined by the transformation:

$$Curr_{i+1} = Curr_i \boxplus D[H_i]$$

where H_i dynamically influences the swaps within D and D^{-1}. This process ensures that no static patterns emerge, maintaining high entropy across all encryption steps. Consequently, the resulting ciphertext remains statistically unpredictable, with entropy characterized by:

$$Entropy(Z_i) = Entropy(H_i) + Entropy(D[P_i]) \qquad (5)$$

This guarantees resistance against frequency analysis, correlation attacks, and other statistical cryptanalysis techniques. The security of RPC is built upon three fundamental principles:

- Uniform Mapping: The array D ensures an even distribution of P_i to $D[P_i]$ over the range {0, 1, ..., 255}.
- Entropy Injection: The sequence H_i introduces additional pseudorandom entropy into both $D[P_i]$ and D^{-1}.
- Diffusion: D^{-1} propagates this entropy throughout the ciphertext space Z_i, maximizing diffusion.

Because H_i and D evolve dynamically during encryption, reverse-engineering these transformations without access to the secret internal state is infeasible. Given the encryption transformation:

$$Z_i = D^{-1}[H_i \oplus D[P_i]]$$

an adversary attempting to recover P_i from Z_i without knowing H_i, D, or D^{-1} must solve:

$$P_i = D^{-1}[H_i \oplus D[Z_i]]$$

This requires computing $H_i \oplus D[Z_i]$ which depends on unknown values H_i and $D[Z_i]$ and then applying D^{-1} to recover P_i. Without full knowledge of D, this computation becomes infeasible.

The attacker faces an exponential complexity challenge: reconstructing D requires brute-forcing 256! possible permutations, while determining H_i involves testing 256^3 = 16,777,216 possible values per encryption step. The total computational complexity of an attack is:

$$O(256! \times 256^3)$$

which is practically infeasible. Even if an attacker gains access to multiple plaintext-ciphertext pairs, the unpredictable nature of H_i and the dynamic permutations in D and D^{-1} prevent any viable decryption strategy. RPC achieves robust security due to:

- An exceptionally large key space ($256! \approx 2^{1,684}$), making brute-force attacks impractical.
- Strong non-linearity, introduced by XOR operations and dynamic permutations, which eliminate linear cryptanalysis paths.
- High entropy and resistance to brute-force, statistical and differential attacks.

By combining randomized permutations with adaptive transformation structures, RPC maintains its encryption security even against advanced attack models, including post-quantum threats.

5.2. Resistance to main cryptographic attacks

- ***Brute Force:***

To recover D, D^{-1}, and the encryption output, an attacker must evaluate all possible configurations of these components. The encryption formula is: $Z_i = D^{-1}[((D^{-1}[i] \oplus D[Curr_i]) \boxplus (D[Curr_i] \oplus Curr_i)) \oplus D[P_i]]$. where H_i is defined as: $H_i = (D^{-1}[i] \oplus D[Curr_i]) \boxplus (D[Curr_i] \oplus Curr_i)$. This interaction between D, D^{-1} and $Curr_i$ results in $256^3 = 16{,}777{,}216$ possible values for H_i. Notably, each byte value of H_i appears exactly 256 times across all combinations, ensuring statistical uniformity and strengthening resistance against statistical attacks. The total number of possible configurations per encryption step is: Configurations per step = $256! \times 16{,}777{,}216$, where 256! represents all possible permutations of D and D^{-1} and 16,777,216 accounts for the possible combinations of H_i in each encryption step. where 256! represents all possible permutations of D and D^{-1}, and 16,777,216 accounts for the possible combinations of H_i at each step. Over es encryption steps, the complexity scales as: $O(256!^{es} \times 256^{3es})$. Given that $256! \approx 2^{1,684}$, brute-force attacks are computationally infeasible even for a single encryption step ($es = 1$). For $es = 10$, the complexity increases exponentially, making brute-force attacks impractical even with quantum computing advancements.

- ***Known Plaintext Attack:***

In a known plaintext attack, the attacker has access to plaintext-ciphertext pairs P_i and Z_i and attempts to deduce the internal states of D and D^{-1} and H_i. Recovering H_i requires knowledge of D and D^{-1} and reversing the computation:

$$Z_i = D^{-1}[\, H_i \oplus D[P_i]\,].$$

The attacker cannot isolate H_i or $D[P_i]$ without access to D and D^{-1}. The non-linear relationship between P_i, Z_i and H_i makes reverse computation infeasible. In this attack, the attacker has access to plaintext-ciphertext pairs (P_i , Z_i) and seeks to deduce the internal states D, D^{-1} and H_i. Using the encryption formula:

$$Z_i = D^{-1}[\, ((D^{-1}[i] \oplus D[Curr_i]) \boxplus D[Curr_i] \oplus Curr_i)) \oplus D[P_i]\,]$$

The attacker must Reverse the permutation of D^{-1} and also isolate $D[P_i]$ by undoing the XOR and addition operations: $D[Pi] = ((D^{-1}[i] \oplus D[Curr_i]) \boxplus (D[Curr_i] \oplus Curr_i)) \oplus D^{-1}[Z_i]$. Solving for P_i requires: $P_i = D^{-1}[D[P_i]]$. With $256^3 = 16{,}777{,}216$ possible values for H_i, each occurring 256 times, the computational complexity makes reversing the encryption process infeasible without full knowledge of the secret states.

- ***Chosen Plaintext Attack:***

In a chosen plaintext attack, the attacker selects a plaintext P_i and observes the corresponding ciphertext Z_i. The encryption transformation:

$$Z_i = D^{-1}[\, ((D^{-1}[i] \oplus D[Curr_i]) \boxplus D[Curr_i] \oplus Curr_i)) \oplus D[P_i]\,]$$

ensures that Z_i is tightly linked to the current state of D , D^{-1} and $Curr_i$. Since D , D^{-1} and H_i are updated dynamically after each step, the mappings cannot be reused or predicted.

Even if the same plaintext P_i is encrypted multiple times, the ciphertexts remain highly unlikely to repeat due to the $256^3 = 16{,}777{,}216$ possible variations of H_i at each step. This dynamic behavior prevents an attacker from exploiting repeated mappings between plaintexts and ciphertexts.

- ***Differential Cryptanalysis:***

RPC is resistant to differential cryptanalysis due to the combined use of XOR operations, modular addition and dynamic permutations, which eliminate direct relationships between input differences ΔP_i and output differences ΔZ_i. Let: $\Delta P_i = P_i \oplus P_i'$ represent a difference in plaintexts. The corresponding difference in ciphertexts is: $\Delta Z_i = Z_i \oplus Z_i'$. Since the encryption process introduces pseudo-random updates to D , D^{-1} and H_i, the mapping between ΔP_i and ΔZ_i is highly non-linear. The $256^3 = 16{,}777{,}216$ possible possible combinations of Hi, combined with the dynamic updates of D, D^{-1} and $Curr_i$, ensure that ΔZ_i is non-linearly related to ΔP_i. This eliminates predictable differential patterns that could be exploited in an attack.

- ***Side-Channel Attacks***

A. Definition of Side-Channel Attacks

Side-channel attacks exploit indirect information, such as execution time, power consumption and electromagnetic radiation, to retrieve sensitive information. The main types of side-channel attacks that affect cryptographic algorithms include:

- Timing Attacks: These attacks exploit variations in the time taken by an algorithm to process different inputs. By analyzing these timing differences, an attacker can infer secret data, such as encryption keys [44].
- Cache Attacks: These attacks target the interaction of cryptographic software with the CPU's cache hierarchy. When an attacker can observe cache hits and misses, they can extract information about the secret key used in cryptographic operations [45].
- Speculative Execution Attacks: Techniques such as Spectre and Meltdown exploit the speculative execution mechanism of modern processors, where instructions are executed out of order, potentially allowing attackers to access protected memory locations [46].

B. Vulnerabilities of RPC to Side-Channel Attacks

The RPC algorithm uses dynamically permuted arrays (D, D^{-1}) for encryption and these operations involve frequent memory lookups. These memory accesses can lead to leakage of sensitive information via side-channel attacks, such as cache timing and branch prediction. Without mitigation techniques, an attacker could potentially recover the secret key by analyzing these leaks [47].

C. Mitigation Techniques for RPC

To protect RPC from side-channel attacks, the following mitigation techniques are recommended:

- Constant-Time Implementations: Ensuring that the algorithm's operations are executed in constant time, regardless of the input data, reduces the likelihood of leaks through timing attacks. This has been shown to significantly increase resistance against side-channel attacks in cryptographic systems [48].
- Masking: Masking techniques introduce random values into intermediate results, making it harder for an attacker to gain useful information from side-channel leaks. This technique has been applied successfully in several cryptographic algorithms to secure sensitive data [49].
- Shuffling: Randomizing the order of operations within the algorithm can make it more difficult for an attacker to predict leakage patterns based on the execution sequence [50].
- Speculative Execution Barriers: Using instructions such as LFENCE (Intel) and DSB SY (ARM) can prevent speculative execution from leaking information through unintentional data paths, as demonstrated in research focused on protecting cryptographic implementations [51].

The application of these mitigation techniques is crucial for ensuring that the RPC algorithm remains secure against side-channel attacks. While these techniques may introduce some performance

6.0. CONCLUSIONS

This paper introduces the Random Position Cipher (RPC), a novel encryption algorithm that combines high-speed encryption with built-in error detection and correction. By leveraging dynamically permuted arrays and a structured blend of substitution, XOR operations and controlled randomness, RPC achieves both strong security and computational efficiency. Performance evaluations show that RPC encrypts data five to six times faster than AES-NI and ChaCha20 while maintaining robust cryptographic properties.

Furthermore, the RPC-256 and RPC-64 variants have successfully passed the GLI-11 and GLI-19 randomness evaluation certifications, confirming their suitability for real-world deployment. Future research directions include hardware acceleration optimizations and post-quantum cryptographic adaptations to enhance both security and efficiency. These advancements could position RPC as a compelling alternative for high-performance encryption across various application domains.

REFERENCES

[1] K. Raut, C. Katkar, "A Comprehensive Review of Cryptographic Algorithms," *International Journal for Research in Applied Science & Engineering Technology (IJRASET)*, 9(XII), 1750-1756, 2021.

[2] National Institute of Standards and Technology (NIST). Advanced Encryption Standard (AES), FIPS PUB 197, Federal Information Processing Standards Publication, November 26, 2001, updated May 9, 2023, U.S. Department of Commerce, https://doi.org/10.6028/NIST.FIPS.197-upd1.

[3] Daniel J. Bernstein, *ChaCha, a variant of Salsa20*, Department of Mathematics, Statistics, and Computer Science, University of Illinois at Chicago, 2008. Retrieved from https://eprint.iacr.org/2008/165.pdf.

[4] Sabitha, S., and Binitha, V. Nair, *Survey on Asymmetric Key Cryptographic Algorithms*, International Journal of Scientific Research in Science, Engineering and Technology, vol. 7, no. 2, pp. 404-408, 2020. doi: 10.32628/IJSRSET207292.

[5] Rivest, R. L., Shamir, A., and Adleman, L., *A Method for Obtaining Digital Signatures and Public-Key Cryptosystems*, Communications of the ACM, vol. 21, no. 2, pp. 120-126, 1978. doi: 10.1145/359340.359342.

[6] Brown, D. R. L., Standards for Efficient Cryptography SEC 1: Elliptic Curve Cryptography (Version 2.0), Certicom Research, 2009.

[7] Bernstein, D. J., & Lange, T., *Post-quantum cryptography*, *Nature*, 549(7671), 188-194, 2017. https://doi.org/10.1038/nature23461.

[8] I. N. John, P. W. Kamaku, D. K. Macharia, and N. M. Mutua, *Error Detection and Correction Using Hamming and Cyclic Codes in a Communication Channel*, in: Pure and Applied Mathematics Journal, vol. 5, no. 6, pp. 220-231, 2016. https://doi.org/10.11648/j.pamj.20160506.17

[9] W. R. Corliss, *A History of the Deep Space Network*, NASA CR-151915, National Aeronautics and Space Administration, Washington, D.C., November 1, 1976. Available at: Legacy CDMS, Accession No. 77N14188.

[10] Clerk, A.A., Devoret, M.H., Girvin, S.M., Marquardt, F., & Schoelkopf, R.J. (2010). Introduction to quantum noise, measurement and amplification. *arXiv:1004.4539*.

[11] S. Chiţu, D. C. Vasile, I. D. Trămândan, P. Svasta, "Key Expansion in Cryptographic Systems," in: *Proc. of the 26th IEEE International Symposium for Design and Technology in Electronic Packaging (SIITME)*, Pitesti, Romania, 2020, pp. 202–205.

[12] Zhang LY, Zhang Y, Liu YS, Yang AJ, Chen G (2017) Security analysis of some diffusion mechanisms used in chaotic ciphers. *International Journal of Bifurcation and Chaos* 27(10):1750155. https://doi.org/10.1142/S0218127417501553.

[13] Jamel S, Deris MM (2008) Diffusive primitives in the design of modern cryptographic algorithms. *International Conference on Computer and Communication Engineering (ICCCE)*, pp. 1–5. https://doi.org/10.1109/ICCCE.2008.4580696.

[14] Author(s) (2016) *Chapter 5: Diffusion in Stream Ciphers*. In: [Title of the Book or Conference Proceedings]. Publisher, pp. 1–5. Corpus ID: 208052530.

[15] Mohamed K, Pauzi MNM, Ali FH, Ariffin S (2022) Analyse on avalanche effect in cryptography algorithm. *European Proceedings of Social and Behavioural Sciences*, 2022(10):57. doi:10.15405/epms.2022.10.57.

[16] Upadhyay D, Gaikwad N, Zaman M, Sampalli S (2022) Investigating the avalanche effect of various cryptographically secure hash functions and hash-based applications. *IEEE Access*, 10: 3215778. doi:10.1109/ACCESS.2022.3215778.

[17] Coskun B, Memon N (2006) Confusion/Diffusion capabilities of some robust hash functions. *Annual Conference on Information Sciences and Systems*. https://doi.org/10.1109/CISS.2006.286645

[18] Dehnavi SM, Mahmoodi Rishakani A, Mirzaee Shamsabad MR, Maimani H, Pasha E (2016) Cryptographic properties of addition modulo 2^n. *IACR Cryptology ePrint Archive*. Corpus ID: 18863587

[19] Katos V, Doherty BS (2007) Exploring confusion in product ciphers through regression analysis. *Information Sciences* 177(7):1519–1526. https://doi.org/10.1016/J.INS.2006.09.017.

[20] L. O'Connor, A. Klapper, "Algebraic Nonlinearity and Its Applications to Cryptography," *Journal of Cryptology*, 7(4), 213–227, 1994.

[21] R. A. Fisher and F. Yates. Statistical tables for biological, agricultural and medical research, pages 26–27. Oliver & Boyd, Third edition, 1948.

[22] D. E. Knuth. In The Art of Computer Programming, Volume 2: Seminumerical Algorithms. Addison-Wesley Longman Publishing Co., Inc., Third edition, 1997.

[23] Golić, J.D.: Linear Cryptanalysis of Stream Ciphers. In: Preneel, B. (ed.) Fast Software Encryption. FSE 1994. Lecture Notes in Computer Science, vol. 1008, pp. 154–169. Springer, Berlin, Heidelberg (1995). https://doi.org/10.1007/3-540-60590-8_13.

[24] E. Biham, O. Dunkelman, "Differential Cryptanalysis in Stream Ciphers," *IACR Cryptology ePrint Archive*, 2007:218 (2007).

[25] Verdult, R.: Introduction to Cryptanalysis: Attacking Stream Ciphers. Manuscript, Radboud University Nijmegen (2015). Available at: https://www.cs.ru.nl/~rverdult/Introduction_to_Cryptanalysis-Attacking_Stream_Ciphers.pdf.

[26] I. Honkala, T. Laihonen, "On the Probability of Undetected Error," in: *Proc. of the 2000 IEEE International Symposium on Information Theory (ISIT)*, Sorrento, Italy, 2000, pp. 254. https://doi.org/10.1109/ISIT.2000.866552.

[27] Y.-Q. Shi, et al., "Interleaving for Combating Bursts of Errors," *IEEE Circuits and Systems Magazine*, 4, 29–42, 2004.

[28] Lipmaa, H., Rogaway, P., Wagner, D.: Comments to NIST Concerning AES Modes of Operations: CTR-Mode Encryption. In: First NIST Workshop on Modes of Operation, 2000.

[29] Bernstein, D.J.: ChaCha, a Variant of Salsa20. Technical Report, University of Illinois at Chicago (2008). Available at: https://cr.yp.to/chacha/chacha-20080128.pdf

[30] Viega, J., Messier, M., Chandra, P.: Network Security with OpenSSL: Cryptography for Secure Communications. O'Reilly Media, Sebastopol (2002).

[31] Moreira, J.C., Farrell, P.G.: Essentials of Error-Control Coding. Wiley, Chichester (2006).

[32] Ding, C., Li, C.: BCH Cyclic Codes. *Discrete Mathematics*, 347, 113918, 2024.

[33] Shokrollahi, Amin. “LDPC Codes: An Introduction.” (2004).

[34] Gilbert, E.N.: Capacity of a Burst-Noise Channel. *Bell System Technical Journal*, 39, 1253–1265, 1960.

[35] Shi YQ, Zhang XM, Ni ZC, Ansari N (2002) Ensuring data fidelity: A number of random error correction codes (ECCs) for combating bursts of errors. *IEEE Transactions on Information Theory*, 48(2): 428-457. doi:10.1109/TIT.2002.1002828.

[36] Conway, T.: Galois Field Arithmetic over GF(p^m) for High-Speed/Low-Power Error-Control Applications. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 51, 709–717, 2004.

[37] Bhaskar, R., et al.: Efficient Galois Field Arithmetic on SIMD Architectures. In: *Proc. of the ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*, 2003.

[38] Rukhin A, Soto J, Nechvatal J, Smid M, Barker E, Leigh S, Levenson M, Vangel M, Banks D, Heckert A, Dray J, Vo S, Bassham LE III (2010) A statistical test suite for random and pseudorandom number generators for cryptographic applications. Revised April 2010. National Institute of Standards and Technology (NIST), Special Publication 800-22.

[39] Sleem L, Couturier R (2020) TestU01 and Practrand: Tools for a randomness evaluation for famous multimedia ciphers. Multimedia Tools and Applications 79(33-34):24075–24088. doi:10.1007/s11042-020-09134-3.

[40] Thangadurai, K.R., Poongodi, V.: A Review on Ron’s Cryptographic Algorithms. *International Journal of Engineering Research and Technology*, 3, 2014.

[41] Sleem, L., Couturier, R.: TestU01 and Practrand: Tools for a Randomness Evaluation for Famous Multimedia Ciphers. *Multimedia Tools and Applications*, 79(33-34), 24075–24088 (2020). https://doi.org/10.1007/s11042-020-09038-4

[42] Gaming Laboratories International (2018) GLI-11: Gaming devices standard, version 3.0. Gaming Laboratories International, New Jersey.

[43] Gaming Laboratories International (2017) GLI-19: Interactive gaming systems, version 3.0. Gaming Laboratories International, New Jersey.

[44] Osvik, A., Shamir, A., & Tromer, E. (2006). Cache attacks and countermeasures: The case of AES. *Advances in Cryptology–CRYPTO 2006*. Springer.

[45] Genkin, D., Shamir, A., & Tromer, E. (2014). RSA key extraction via low-bandwidth acoustic cryptanalysis. *Advances in Cryptology–CRYPTO 2014*. Springer.

[46] Kiriansky, V., & Waldspurger, C. (2018). Speculative buffer overflows: Attacks and defenses. *IEEE Security & Privacy*.

[47] Bernstein, D. J. (2005). Cache-timing attacks on AES. *University of Illinois at Chicago, Technical Report*.

[48] Weißbart, M., & Güneysu, T. (2019). Towards side-channel resistant implementations of AES on embedded systems using ARX operations. *IEEE Transactions on Computers, 69(2), 153-165*.

[49] Costan, V., & Devadas, S. (2016). Intel SGX explained. *IACR Cryptology ePrint Archive, 2016, 86*.

[50] Kocher, P. (1996). Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems. *Advances in Cryptology–CRYPTO 1996*. Springer.

[51] Kiriansky, V., & Waldspurger, C. (2018). Speculative buffer overflows: Attacks and defenses. *IEEE Security & Privacy*.